

Short Lecture: Optimization

Kristopher Micinski

Syracuse University

CIS531, Fall 2025



- ✱ So far in this class, we have discussed how to compile a ***complete language***
 - ✱ Arithmetic/let, bool/if, vectors/set!/while, functions/lambdas
- ✱ Issue: code is often obviously inefficient in very clear ways
- ✱ Today I will briefly discuss a set of optimizations which might be low-hanging fruit

Partial Evaluation

- * Idea: I can view the program as having a “dynamic” portion (user / external inputs), and a “static” portion (propagation of data within the program)
- * I can mush the static stuff together as much as I want as long as I respect functional correctness:

```
(let ([y (read)])  
  (let ([_ (void)])  
    (let ([x y])  
      (let ([z x])  
        (+ z z))))
```



```
  (let ([y (read)])  
    (+ y y))
```


- * I can always eliminate expressions like this...

$$(\text{let } ([_ \text{ (void) }]) \text{ e}) \longrightarrow \text{e}$$

- * Variable swapping can be eliminated via substitution...

$$(\text{let } ([x \ y]) \text{ e}) \longrightarrow \text{e } [x \mapsto y]$$

- * Could also do...

$$(\text{let } ([x \ e']) \text{ e}) \longrightarrow \text{e } [x \mapsto e']$$

- * But doing this might make the term **expand**

Implementing Partial Evaluation

- * Lots of ways to implement / conceptualize partial evaluation
 - * Mostly I am not qualified to cover these, will cover specific instance
- * Basic intuition: run rules to a fixed point
 - * Clear issue: expansive rewrites imply possible nontermination
 - * Leverage a **memo table** to ensure termination
- * Can employ **binding-time analysis** (BTA), which is a very simple abstract interpretation over the two-point lattice {Static < Dynamic}
 - * Static is $\perp$, Dynamic is $\top$
 - * Followed by **specialization**, which consults the BTA to fold together the static computations and emit residual code for the dynamic ones

- * Just like variables, it makes sense to do constant propagation because doing so makes the overall term smaller...

$$(\text{let } ([x \ n]) \ e) \longrightarrow e \ [x \mapsto n]$$

- * When we can statically compute e_0 / e_1 ...

$$(+ \ e_0 \ e_1) \wedge e_0/e_1 \ \text{static} \longrightarrow \text{Residualize } e_0+e_1$$

- * When we are applying a statically-known lambda, inline it...

$$((\lambda \ (x) \ e) \ e-a) \longrightarrow e \ [x \mapsto e-a]$$

Control-Flow Analysis (CFA)

- * Control-flow analysis (CFA) answers the question: “what possible set of *lambdas* does each function application (in the program) target?”
- * Every function application *expr* targets some set of *syntactic lambdas*
- * CFA often very important in functional languages
 - * Need to know CFA to construct “call graph”
 - * This is ***significantly easier*** in languages like C *if you ignore function pointers* (i.e., no higher-order / first-class functions)
 - * In these settings, call graph is syntactically apparent
- * Will begin by discussing ***context-insensitive CFA***

Question: CFA

What specific lambda reaches each call site?

```
(define d ( $\lambda^0$  (x y) x))  
(define f ( $\lambda^1$  (z x) z))  
(define g ( $\lambda^2$  (x) d))  
(define a ( $\lambda^4$  (b) ( $\lambda^3$  (x) (x b))))  
(define id ( $\lambda^5$  (x) x))
```

```
(define e0 (d f d))  
(define e1 (e0 id f))  
(define e2 (e1 a))  
(define e3 (e2 g))  
(e3 id)
```


Question: CFA

What specific lambda reaches each call site?

```
(define d ( $\lambda^0$  (x y) x))  
(define f ( $\lambda^1$  (z x) z))  
(define g ( $\lambda^2$  (x) d))  
(define a ( $\lambda^4$  (b) ( $\lambda^3$  (x) (x b))))  
(define id ( $\lambda^5$  (x) x))
```

```
(define e0 (d f d)) ;; {0} reaches d  
(define e1 (e0 id f)) ;; {1} reaches e0  
(define e2 (e1 a)) ;; {5} reaches e1  
(define e3 (e2 g)) ;; {4} reaches e2  
(define e4 (e3 id)) ;; {3} reaches e3
```

Control-Flow Analysis (CFA) and inlining

- * We can only do static lambda inlining in the cases where we have a literal, static lambda being applied to some argument (a so-called “left-left” lambda, in Scheme speak)
- * Often, we simply can’t do this... (e.g., if we even have a variable)
- * But, control-flow analysis is also propagating **data** flows through the analysis, and the result of CFA **might** be able to tell us when we can inline:
- * Let’s say we have (e-f e-args ...)
- * If we run CFA and determine e-f points at **one unique lambda** $\{(\lambda (x) e)\}$:
 - * Then we know that it’s really $((\lambda (x) e) \text{ e-args } \dots)$, and can **inline** it

Context Sensitivity, or lack thereof...

- * In the most basic case, we consider a **context-insensitive** analysis
 - * A context-insensitive analysis strategically loses track of the calling context—each function (`foo x`) is analyzed as if `x` could be **any** incoming value

```
;; context-insensitive id tracks x = {f, g}
(define (id x) x)
(define (f x) (/ 1 x))
(define (g x) (/ x 1))
```

```
(define h (id f)) ;; {f,g}
(define l (id g)) ;; {f,g}
(l 0) ;; *NOT SAFE*, ({f,g} 0)
```

The call to `l` here is **unsafe**, because of the loss of context sensitivity in the analysis of `id`

The analysis of `id` conflates `f` and `g`, so that the return value from `id` is the **flow set** `{f,g}`, “either the function `f` or the function `g`”

Because it **appears it could be `f`**, we throw an error (division by zero)

But this is a **false positive**

Calculating CFA

- * OCFA asks: “for each binding, what is the **set of lambdas**, which may be bound to this callsite, in any possible context from the program.”
- * Recall the last slide: mush together all calls to function f
- * What we have not said: how to **calculate** CFA
 - * Can formulate a set of **flow analysis equations** for CFA, just as we did for the Kildall-style analyses
 - * Generally beyond the scope of this class—

Escape Analysis

- * In our assignment conversion pass, we box **every local variable**, and put it on the heap, which of course has major downsides...
- * Most importantly, we pay the price of an allocation, which is huge
 - * We would like to be able to put the variable on the stack: 0 allocation
- * **But**, we can **only** stack-allocate a variable if we **know** it cannot **escape** the function's frame (dynamic extent)

For example... let's say we're compiling the outer lambda:
we can see that y “escapes” the lambda's dynamic extent

However, x/y cannot escape the lambda's frame
For the **inner** lambda, a/b/c do not escape

```
(λ (x y z)
  (λ (a b)
    (let ([c a])
      (+ c (+ b y))))))
```

Roadmap for our optimizing our compiler...

- * Some low-hanging fruit that would make our compiler much better...
- * Register allocation (need this)
- * Eliminating obviously-dead code (`let ([_ (void)]) ...`)
 - * Also remove renamed variables via substitution (`let ([x y]) e`) $\Rightarrow$ `e[x->y]`
- * Escape analysis to determine which variables (might) escape
 - * Rewriting assignment conversion based on result
 - * Assignment conversion adds a **massive** amount of overhead, mostly unnecessary (but a concession for making lifetimes simple w/ set!)
- * More partial evaluation: coalescing blocks, etc.
 - * Situating blocks immediately next to each other if there is a single jump
 - * Just fall through!
- * Can think of each of these as potential projects forming “compilers ii” course

Optimization in Real Compilers

- * Real compilers employ optimizations at various stages, with some concern about interplay between passes
- * In practice, optimizations must often run in a loop / to a fixed point
- * Bulk of optimization typically happens on some ANF/SSA/etc.-style IR
- * SSA/ANF assumptions simplify reasoning a lot, most analyses need this
- * How much / many analyses depends on optimization level (-O0, -O3, etc.)
- * LLVM (will discuss later in class...): clang emits LLVM IR
- * LLVM then runs a large set of **passes** to a fixed point, which either (a) add analysis information or (b) perform code transformation based on the result of a previous analysis.
- * (E.g., mem2reg <https://longfangsong.github.io/en/mem2reg-made-simple/>)